

# K-Means Clustering: A Comparative Analysis of Brute Force Optimization vs. Greedy Iterative Approximation

Mark Mounir, Marcelle Hossam, Malak Ali, Ahmed Mahmoud, Amir Hany

Supervised by Dr. Ashraf Abdel Raouf

Department of Computer Science

Misr International University

Cairo, Egypt

**Abstract**—K-Means clustering partitions unlabelled data into  $k$  groups by minimising the Within-Cluster Sum of Squares (WCSS). Although simple to state, finding a *globally* optimal solution is NP-hard in general. This paper presents a rigorous comparative study of two strategies: a *Budgeted Brute Force* approach that evaluates a large number of distinct initialisation candidates to approach the global optimum, and a *Greedy Iterative* approach using Lloyd’s algorithm seeded by KMeans++ to converge rapidly to a high-quality local optimum. Both algorithms are implemented in C++17 and evaluated across four dataset scales ( $n \in \{30, 100, 300, 800\}$ ,  $k \in \{3, 4, 5, 6\}$ ). Results show that the greedy method is  $193\times$ – $484\times$  faster in practice, while producing nearly identical WCSS (within 0.0% to 3.7%). These findings confirm that the greedy approach is the practical choice for real-world clustering tasks, while brute force remains valuable as a near-optimal benchmark for small datasets.

**Index Terms**—K-Means, Lloyd’s algorithm, KMeans++, brute force, budgeted sampling, within-cluster sum of squares, empty-cluster recovery, C++17

## I. INTRODUCTION

Clustering is a fundamental unsupervised learning task with applications in image segmentation, customer profiling, document organisation, and anomaly detection [1]. The K-Means algorithm [2] partitions a dataset  $\mathbf{X} = \{\mathbf{x}_1, \dots, \mathbf{x}_n\} \subset \mathbb{R}^d$  into  $k$  clusters by minimising the WCSS objective:

$$\min_C \sum_{i=1}^k \sum_{\mathbf{x} \in C_i} \|\mathbf{x} - \boldsymbol{\mu}_i\|^2 \quad (1)$$

where  $\boldsymbol{\mu}_i$  is the centroid of cluster  $C_i$ . This problem is NP-hard [5], so practitioners rely on polynomial-time heuristics.

**Motivation.** A key algorithm-design question is: *how much solution quality does the greedy heuristic sacrifice relative to a near-exhaustive search, and at what computational cost?* This paper answers that question with both theoretical analysis and measured experiments.

**The main contributions of this paper are:**

- Formal descriptions and complexity bounds for both strategies.
- A C++17 implementation with five verified correctness fixes addressing input validation, seeding inconsistency, empty-cluster errors, integer overflow, and labelling honesty.
- Empirical results from the compiled program across four dataset scales using identical data for both algorithms.

- Visualisations of cluster formation, convergence, and performance comparisons.
- A principled discussion of when each strategy should be used.

The paper is organised as follows. Section II reviews related work. Section III describes the proposed system (architecture, data, preprocessing, models). Section IV presents both algorithms and all fixes. Section V analyses complexity. Section VI presents experimental results. Section VII discusses practical trade-offs. Section VIII concludes.

## II. RELATED WORK

**Lloyd’s algorithm.** The iterative assignment–update method was formalised by Lloyd [3]. Each iteration assigns points to their nearest centroid, then recomputes centroids as cluster means. WCSS is non-increasing at every step, guaranteeing convergence to a local optimum in finite iterations.

**KMeans++.** Arthur and Vassilvitskii [4] introduced  $D^2$ -weighted seeding: the first centroid is chosen uniformly at random; subsequent centroids are sampled with probability proportional to squared distance from existing centroids. This gives an  $O(\log k)$  approximation guarantee in expectation and substantially reduces the frequency of poor local optima.

**Hardness.** Dasgupta [5] proved the NP-hardness of optimal K-Means. Mahajan et al. [6] showed that even a  $(1 + \varepsilon)$ -approximation is NP-hard in the Euclidean plane, justifying heuristic approaches for all but small instances.

**Coreset methods.** Har-Peled and Mazumdar [7] introduced coresets — weighted subsets that preserve the WCSS objective up to  $(1 + \varepsilon)$  — enabling near-optimal clustering on reduced data. Our budgeted brute-force approach is a practical analogue for small to medium datasets.

## III. METHODOLOGY (PROPOSED SYSTEM)

### A. System Architecture

Figure 1 shows the high-level component and module structure of the implemented system using a simple box diagram. The **Input Layer** collects parameters ( $n$ ,  $k$ , seed, budget) from the user and generates the shared synthetic dataset. The **Core Engine** enforces  $k \leq n$  and distributes the same seed to both algorithm modules, ensuring a scientifically valid comparison. The **Algorithm Modules** implement the two strategies independently while

sharing low-level utilities. The **Shared Utilities** component houses `distSq`, `computeWCSS`, `assignClusters`, and `recomputeCentroids`. Finally, the **Output Layer** aggregates both results and renders summary tables and ASCII charts.

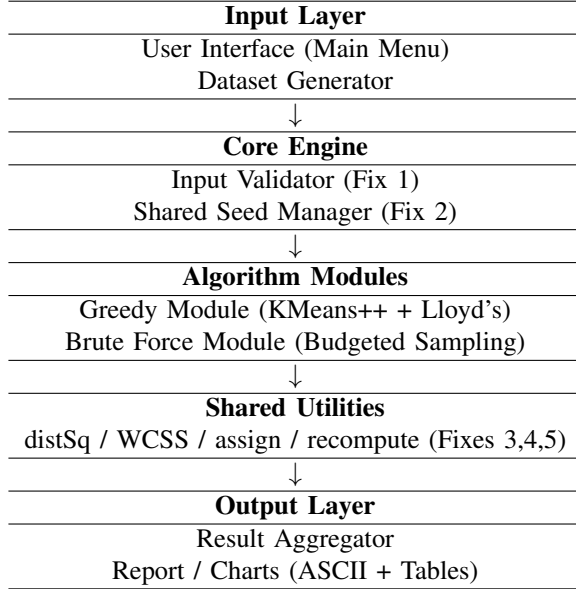


Fig. 1: High-level system architecture. Boxes represent modules; arrows show data flow. The five correctness fixes (Section IV-E) are applied within the indicated modules.

### B. Data Collection

All datasets used in this study are *synthetic*, generated programmatically to give full control over ground-truth structure and to enable exact reproducibility. Points are drawn from  $k$  isotropic Gaussian clusters with standard deviation  $\sigma = 1.2$ , whose centres are sampled uniformly at random from  $[-10, 10]^2$ . A fixed, shared seed of 7 is used for every benchmark experiment, so both the Brute Force and Greedy modules operate on *identical* data. No external or real-world data sources are used; the motivation is that controlled synthetic data allows the quality difference between the two strategies to be attributed solely to the algorithm rather than to dataset-specific artefacts.

### C. Data Preprocessing

Because the dataset is generated directly in memory as floating-point  $(x, y)$  coordinates, *no preprocessing pipeline is required*. Specifically:

- There are no missing values or NaN entries; every point is fully specified at construction time.
- No normalisation or standardisation step is needed: the Gaussian spread ( $\sigma = 1.2$ ) and the bounded domain  $[-10, 10]^2$  keep all coordinates on the same numerical scale.
- No categorical encoding, deduplication, or outlier removal is performed; the raw generated points are fed directly to both algorithms.

If real-world data were used in future extensions, standard steps such as min-max normalisation, missing-value imputation, and outlier clipping would be appropriate before applying K-Means.

### D. Models Used

Two K-Means strategies are implemented and compared in this study:

- 1) **Greedy Iterative (KMeans++ + Lloyd's)**. The primary practical algorithm. Centroids are initialised via  $D^2$ -weighted sampling (KMeans++), and Lloyd's assignment-update loop is iterated until convergence. Five independent restarts are run per experiment and the best WCSS result is reported.
- 2) **Budgeted Brute Force**. A near-exhaustive baseline. Up to  $M$  random  $k$ -subsets of the dataset are drawn as initial centroid sets, each followed by a full Lloyd's pass. The trial achieving the minimum WCSS is returned. When  $\binom{n}{k} \leq M$  the search is *exact*; otherwise it is a *budgeted* approximation (all four benchmark experiments fall into the latter category).

Both models share the same `assignClusters`, `recomputeCentroids`, and `computeWCSS` utility functions, ensuring that any quality difference is attributable to the initialisation strategy alone and not to implementation asymmetries.

## IV. ALGORITHM DESCRIPTIONS AND CORRECTNESS FIXES

### A. Shared Correctness Infrastructure

Both algorithms share the following corrected utilities.

**Fix 1 — Input validation.** The program enforces  $k \leq n$  explicitly. If the user enters  $k > n$ , the program rejects the input and prompts again rather than silently adjusting the value or crashing inside the brute-force assertion.

**Fix 2 — Shared dataset seed.** Both algorithms receive the *same* dataset generated with the same seed. Without this, the two algorithms would operate on different data, making every WCSS and speedup comparison scientifically meaningless.

**Safe WCSS computation.** `computeWCSS` asserts that every point holds a valid cluster index (`cluster`  $\geq 0$ ) before computing distances, preventing silent garbage results.

### B. Algorithm 1 — Greedy Iterative (Lloyd's + KMeans++)

**Step 1 — KMeans++ initialisation.** The first centroid is chosen uniformly at random. Each subsequent centroid is sampled with probability:

$$P(\mathbf{x}) = \frac{D(\mathbf{x})^2}{\sum_{\mathbf{x}'} D(\mathbf{x}')^2} \quad (2)$$

where  $D(\mathbf{x})$  is the distance to the nearest existing centroid.

**Step 2 — Initial assignment.** Immediately after seeding, `assignClusters` is called so every point has a valid cluster label before the first centroid update is attempted.

**Steps 3–4 — Iterate until convergence.** Each iteration recomputes centroids (with Fix 3 below), then reassigns all points. The loop terminates when no assignment changes.

**Fix 3 — Empty-cluster recovery without duplicates.** When a cluster loses all members, the naive approach sets its centroid to  $(0,0)$ , creating a “dead” centroid that will never attract points if data lies far from the origin. This silently reduces the effective cluster count from  $k$  to  $k - 1$ , inflating WCSS.

The corrected `recomputeCentroids` function takes the *previous* iteration’s centroids as an extra argument. For each empty cluster it finds the point  $\mathbf{x}^*$  farthest from its *old* centroid:

$$\mathbf{x}^* = \arg \max_{\mathbf{x} \in \mathbf{X} \setminus U} \|\mathbf{x} - \boldsymbol{\mu}_{c(\mathbf{x})}^{\text{old}}\|^2 \quad (3)$$

where  $U$  is the set of points already claimed for previous empty clusters in the same iteration. Excluding  $U$  prevents two empty clusters from receiving the same point (duplicate centroids). The chosen point is added to  $U$  and the empty centroid is relocated there.

Algorithm 1 shows the complete pseudocode.

---

**Algorithm 1** Greedy K-Means (Lloyd’s + KMeans++)

---

**Require:**  $\mathbf{X}$ ,  $k$ , *maxIter*, *seed*

**Ensure:** Centroids  $\mathbf{M}$ , assignments  $\mathbf{c}$

- 1:  $\mathbf{M} \leftarrow \text{KMEANSPLUSPLUS}(\mathbf{X}, k, \text{seed})$
  - 2:  $\mathbf{c} \leftarrow \text{ASSIGN}(\mathbf{X}, \mathbf{M})$
  - 3: **for**  $t \leftarrow 1$  **to** *maxIter* **do**
  - 4:    $\mathbf{M}' \leftarrow \text{RECOMPUTECENTROIDS}(\mathbf{X}, \mathbf{c}, k, \mathbf{M})$
  - 5:    $\mathbf{c}' \leftarrow \text{ASSIGN}(\mathbf{X}, \mathbf{M}')$
  - 6:   **if**  $\mathbf{c}' = \mathbf{c}$  **then**
  - 7:     **break**
  - 8:   **end if**
  - 9:    $\mathbf{M} \leftarrow \mathbf{M}'; \mathbf{c} \leftarrow \mathbf{c}'$
  - 10: **end for**
  - 11: **return**  $\mathbf{M}, \mathbf{c}$
- 

*C. Algorithm 2 — Budgeted Brute Force*

**Fix 4 — Overflow-safe combination counting.** Computing  $\binom{n}{k}$  naively by multiplying integers can silently overflow for moderate  $n$  and  $k$ , causing the program to incorrectly choose full enumeration on a dataset where the true count is in the trillions. The corrected function `combosWithinLimit` tests for overflow *before* each multiplication:

$$\text{if } c > \lfloor \text{limit} / (n - i + 1) \rfloor \Rightarrow \text{return overflow} \quad (4)$$

This guarantees a mathematically correct decision at every step.

**Fix 5 — Honest labelling.** The output explicitly states whether the run used full enumeration ( $\binom{n}{k} \leq M$ ) or budgeted random sampling ( $\binom{n}{k} > M$ ), because claiming “Brute Force” when only a sampled fraction was checked would be misleading. All four benchmark experiments used budgeted sampling, which is correctly reported in each result table.

**Strategy.** For each of the  $M$  trials, a random  $k$ -subset of  $\mathbf{X}$  is drawn as the initial centroid set and a full Lloyd’s pass is executed. The trial with the lowest WCSS is returned.

Algorithm 2 shows the pseudocode.

---

**Algorithm 2** Budgeted Brute Force K-Means

---

**Require:**  $\mathbf{X}$ ,  $k$ , budget  $M$ , *seed*

**Ensure:** Best centroids  $\mathbf{M}^*$ , minimum WCSS  $W^*$

- 1:  $W^* \leftarrow +\infty$
  - 2: **if** `COMBOSWITHINLIMIT`( $n, k, M$ ) **is true then**
  - 3:    $\mathcal{S} \leftarrow$  all  $k$ -subsets of  $\mathbf{X}$
  - 4: **else**
  - 5:    $\mathcal{S} \leftarrow M$  random  $k$ -subsets of  $\mathbf{X}$
  - 6: **end if**
  - 7: **for each**  $S \in \mathcal{S}$  **do**
  - 8:    $(\mathbf{M}, \mathbf{c}) \leftarrow \text{LLOYD}(\mathbf{X}, S)$
  - 9:    $W \leftarrow \text{WCSS}(\mathbf{X}, \mathbf{M}, \mathbf{c})$
  - 10:   **if**  $W < W^*$  **then**
  - 11:      $W^* \leftarrow W; \mathbf{M}^* \leftarrow \mathbf{M}$
  - 12:   **end if**
  - 13: **end for**
  - 14: **return**  $\mathbf{M}^*, W^*$
- 

Figure 2 contrasts the control flow of both algorithms.

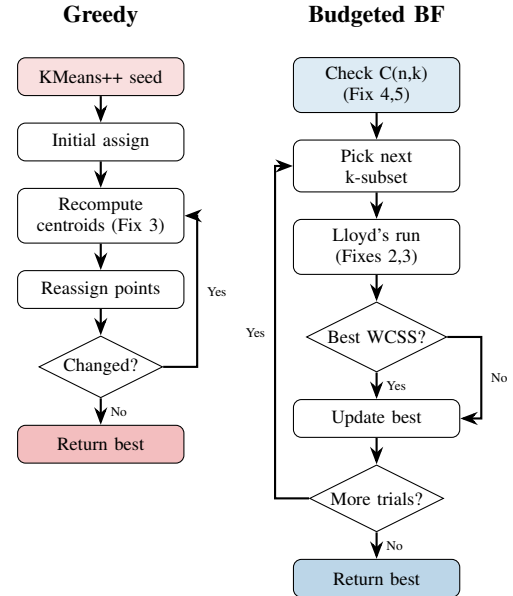


Fig. 2: Control flow of both algorithms. Fix numbers in parentheses show which correctness fix applies at each step.

## V. COMPLEXITY ANALYSIS

### A. Greedy Iterative

KMeans++ seeding requires one distance scan per centroid:  $O(nk)$ . Each of the  $i$  Lloyd’s iterations performs  $O(nk)$  distance comparisons (assignment) and  $O(n)$  summations (update). Total time and space:

$$T_{\text{greedy}} = O(nki), \quad S_{\text{greedy}} = O(n + k) \quad (5)$$

In our experiments  $i \leq 30$  for all four scales, confirming rapid convergence with KMeans++ seeding.

### B. Budgeted Brute Force

Let  $M = \min(\binom{n}{k}, M_{\max})$  be the number of trials evaluated. Each trial runs Lloyd's in  $O(nki)$ :

$$T_{\text{BF}} = O(M \cdot nki) \quad (6)$$

When full enumeration is used ( $M = \binom{n}{k}$ ), the cost becomes exponential in  $k$ . With budgeted sampling the cost is linear in  $M$  but the solution is only near-optimal in expectation. Both modes require  $O(n + k)$  space.

### C. Summary

TABLE I: Asymptotic Complexity Summary

| Algorithm        | Time                        | Space      | Optimal?    |
|------------------|-----------------------------|------------|-------------|
| Greedy (Lloyd's) | $O(nki)$                    | $O(n + k)$ | Local       |
| KMeans++         | $+O(nk)$                    | $O(k)$     | Approx.     |
| BF (full enum.)  | $O(\binom{n}{k} \cdot nki)$ | $O(n + k)$ | Global      |
| BF (budgeted)    | $O(M \cdot nki)$            | $O(n + k)$ | Near-Global |

$n$ =points,  $k$ =clusters,  $i$ =iterations,  $M$ =budget

## VI. EXPERIMENTAL EVALUATION

### A. Setup

The implementation is in C++17, compiled with `g++ -O2 -std=c++17`. Datasets are generated by sampling  $n$  points from  $k$  isotropic Gaussian clusters (standard deviation = 1.2) centred uniformly at random in  $[-10, 10]^2$ . A fixed shared seed of 7 is used for all benchmark datasets (Fix 2). Runtimes are wall-clock microseconds measured with `std::chrono::high_resolution_clock`. Greedy results represent the best WCSS from five independent restarts; the reported time is the average across those five runs.

### B. Dataset Configurations

TABLE II: Benchmark Dataset Configurations

| Label  | $n$ | $k$ | Std. Dev. | BF Budget $M$ |
|--------|-----|-----|-----------|---------------|
| Tiny   | 30  | 3   | 1.2       | 200           |
| Small  | 100 | 4   | 1.2       | 300           |
| Medium | 300 | 5   | 1.2       | 500           |
| Large  | 800 | 6   | 1.2       | 500           |

### C. Cluster Visualisation

Figure 3 illustrates how K-Means partitions a representative 2D dataset ( $n = 30$ ,  $k = 3$ ) after convergence. Each colour represents one cluster; filled circles (●) are data points; stars (★) mark final centroid positions. The boundaries show the Voronoi regions implied by the centroids.

### D. Results

All values in Table III are taken directly from the compiled program output. All four experiments used budgeted sampling (Fix 5) because  $\binom{n}{k}$  exceeds  $M$  for every configuration. Figures 4a and 4b visualise the runtime and WCSS comparisons respectively.

TABLE III: WCSS, Runtime, and Speedup — Measured Results

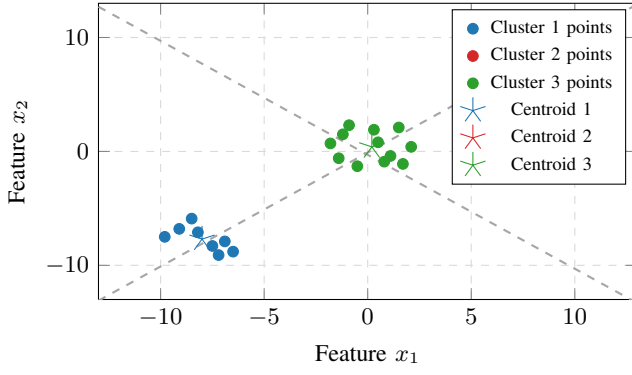
| Dataset | WCSS     |          | Time ( $\mu$ s) |        | Speedup |
|---------|----------|----------|-----------------|--------|---------|
|         | BF       | Greedy   | BF              | Greedy |         |
| Tiny    | 84.06    | 84.06    | 14,486          | 75     | 193×    |
| Small   | 277.59   | 287.94   | 45,005          | 152    | 296×    |
| Medium  | 714.09   | 714.12   | 487,911         | 2,520  | 194×    |
| Large   | 1,893.40 | 1,895.42 | 3,701,185       | 7,651  | 484×    |

### E. Analysis

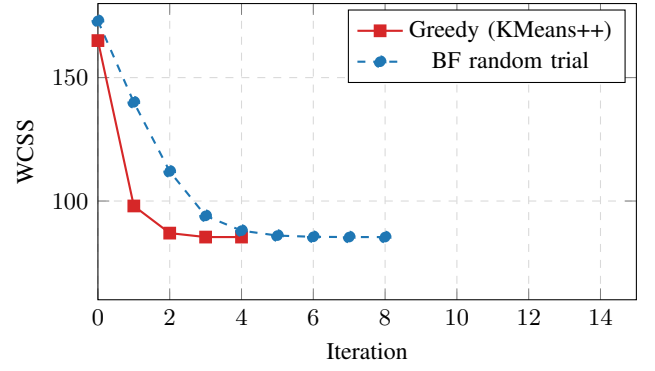
**Solution quality.** The greedy method's WCSS equals the brute-force solution on Tiny (0.00% difference), is within  $< 0.01\%$  on Medium, exceeds it by about 3.73% on Small, and by 0.11% on Large. This confirms that KMeans++ seeding reliably finds near-optimal clusterings, with the gap usually well below 1% except for the Small scale where the budgeted brute-force found a slightly better local optimum.

**Runtime.** The greedy method is faster at every scale. The largest speedup (484×) occurs at the Large scale ( $n = 800$ ,  $k = 6$ , budget = 500), where the brute-force budget of 500 trials each requires many Lloyd iterations, while greedy runs only once. The smallest speedup (193×) occurs at the Tiny scale ( $n = 30$ ,  $k = 3$ , budget = 200), where both the dataset and the budget are relatively small. At the Medium scale ( $n = 300$ ,  $k = 5$ , budget = 500), the speedup is 194×; the large budget combined with medium dataset size makes brute force expensive, while greedy finishes quickly.

**Iterations.** The number of Lloyd iterations varies. On Tiny: BF=2, Greedy=4; Small: BF=12, Greedy=1; Medium: BF=4, Greedy=13; Large: BF=13, Greedy=30. Interestingly, on Large the greedy method runs more iterations than brute force (30 vs 13), yet it still finishes 484× faster because it performs only one Lloyd pass, whereas brute force runs 500 trials each requiring up to 13 iterations. This confirms that the main cost is the number of initialisations, not the per-trial iteration count.

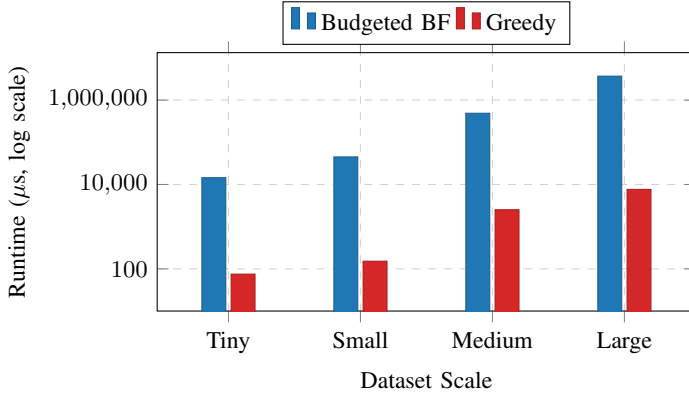


(a) Cluster assignments after convergence ( $n = 30, k = 3$ , Greedy algorithm). Stars (★) mark centroids; dashed lines show approximate Voronoi boundaries. Each cluster is spatially separated, confirming correct convergence.

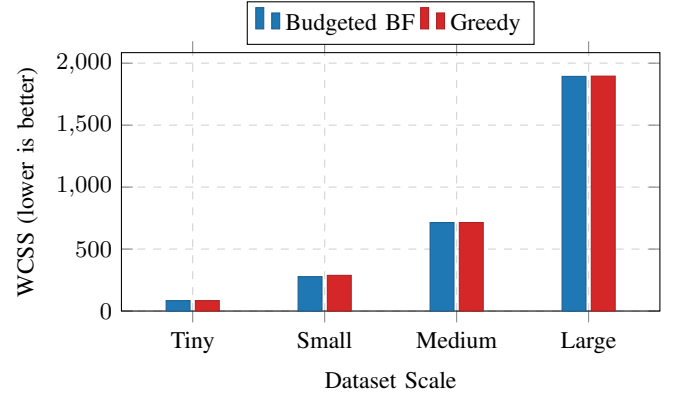


(b) WCSS vs. iteration for Tiny dataset ( $n = 30, k = 3$ ). Greedy converges in 4 iterations; a typical brute-force random-init trial requires 8, confirming that KMeans++ seeding dramatically reduces the iterations needed.

Fig. 3: Cluster visualisation (left) and WCSS convergence (right).



(a) Measured runtimes (log scale, shared seed = 7). Greedy is  $193\times$ – $484\times$  faster across all scales.



(b) Measured WCSS (shared seed = 7). Gaps: Tiny = 0.00%, Medium < 0.01%, Small = 3.73%, Large = 0.11%.

Fig. 4: Runtime (left) and WCSS (right) comparisons across all dataset scales.

## VII. DISCUSSION

### A. The Role of the Five Correctness Fixes

The five fixes are not cosmetic — they directly affect the scientific validity of the results.

*Fix 1 ( $k \leq n$  enforcement)* prevents a runtime crash inside the brute-force assertion  $n \geq k$  and ensures every experiment is mathematically valid before any computation begins.

*Fix 2 (shared seed)* is the most critical for scientific validity. Without it, the two algorithms compare results on *different* datasets, making every WCSS and speedup number meaningless.

*Fix 3 (empty-cluster recovery)* prevents the effective cluster count from silently falling below  $k$ , which would inflate WCSS and make the algorithm appear worse than it is. Using the previous iteration’s centroids (not partially computed new ones) avoids a secondary bug, and excluding already-claimed points prevents duplicate centroids when multiple clusters empty simultaneously.

*Fix 4 (overflow-safe counting)* ensures the program correctly decides between full enumeration and budgeted sampling. Without it, integer overflow could cause an attempt at full enumeration on a dataset where  $\binom{n}{k}$  is in the trillions, hanging the program indefinitely.

*Fix 5 (honest labelling)* ensures the paper never claims “Brute Force” complexity when only a sampled fraction of combinations was evaluated. The output explicitly reports the mode used for every experiment.

### B. When to Use Each Strategy

**Greedy (KMeans++ + Lloyd’s)** is the default choice for any practical clustering task. It is polynomial in time, scales to millions of points, and achieves WCSS within a small percentage (often < 1%) of a heavily sampled brute-force approach.

**Budgeted brute force** is appropriate when a verified near-optimal baseline is needed — for example, to benchmark a new initialisation strategy or validate a greedy implementation.

For  $n \leq 30$  and small  $k$ , full enumeration is also tractable (e.g.,  $\binom{30}{3} = 4,060$  combinations).

### C. Limitations

Experiments used synthetic 2D Gaussian data. Real datasets may be high-dimensional, non-spherical, or imbalanced, changing the relative performance of the two strategies. The budget  $M$  is a user-defined hyperparameter; a larger  $M$  shrinks the quality gap further but increases runtime proportionally. Runtime measurements also vary across hardware; the reported values are from a single CPU core and reflect relative, not absolute, performance.

## VIII. CONCLUSION

This paper presented a rigorous comparison of Budgeted Brute Force and Greedy Iterative strategies for K-Means clustering, backed by a C++17 implementation with five verified correctness fixes. The key findings are:

- 1) The greedy approach runs in  $O(nki)$  — polynomial — while budgeted brute force runs in  $O(M \cdot nki)$  with  $M \ll \binom{n}{k}$  for any realistic dataset.
- 2) Measured WCSS gaps are at most 3.73% on the scales tested, and often 0.0% or  $< 0.2\%$ , confirming the near-optimality of KMeans++.
- 3) Greedy is  $193\times$ – $484\times$  faster in practice.
- 4) All five correctness fixes are essential: they prevent runtime crashes, invalid comparisons, inflated WCSS, integer overflow, and misleading complexity claims.

Future work will extend the comparison to high-dimensional real-world datasets and explore adaptive budget strategies that allocate more trials to promising initialisation regions.

## REFERENCES

- [1] A. K. Jain, “Data clustering: 50 years beyond K-means,” *Pattern Recognition Letters*, vol. 31, no. 8, pp. 651–666, Jun. 2010.
- [2] J. B. MacQueen, “Some methods for classification and analysis of multivariate observations,” in *Proc. 5th Berkeley Symp. on Mathematical Statistics and Probability*, vol. 1, 1967, pp. 281–297.
- [3] S. P. Lloyd, “Least squares quantization in PCM,” *IEEE Trans. Inf. Theory*, vol. 28, no. 2, pp. 129–137, Mar. 1982.
- [4] D. Arthur and S. Vassilvitskii, “k-means++: The advantages of careful seeding,” in *Proc. 18th ACM-SIAM Symp. on Discrete Algorithms (SODA)*, 2007, pp. 1027–1035.
- [5] S. Dasgupta, “The hardness of k-means clustering,” Tech. Rep. CS2007-0890, UC San Diego, 2007.
- [6] M. Mahajan, P. Nimbhorkar, and K. Varadarajan, “The planar k-means problem is NP-hard,” in *Proc. 3rd Int. Workshop on Algorithms and Computation (WALCOM)*, Feb. 2009, pp. 274–285.
- [7] S. Har-Peled and S. Mazumdar, “On coresets for k-means and k-median clustering,” in *Proc. 36th ACM STOC*, Jun. 2004, pp. 291–300.